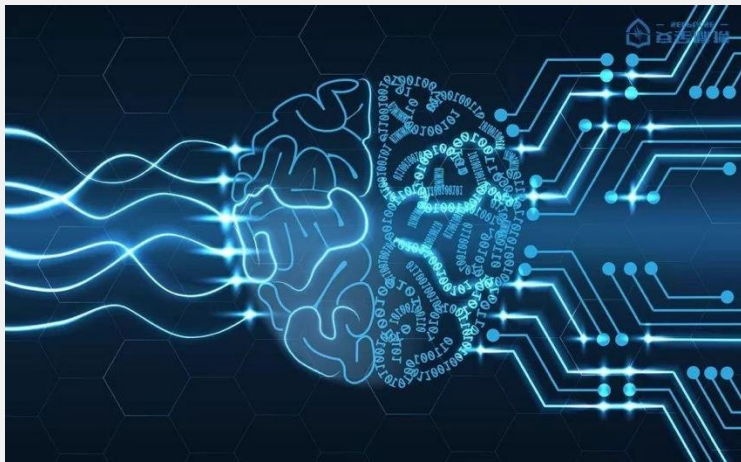




# 机器学习

Machine Learning

授课老师：程骋

办公室：南一楼中315

邮箱：[c\\_cheng@hust.edu.cn](mailto:c_cheng@hust.edu.cn)

# 第十一章、迁移学习

## 目录 CONTENTS

**01** 迁移学习背景

**02** 迁移学习概念

**03** 迁移学习方法

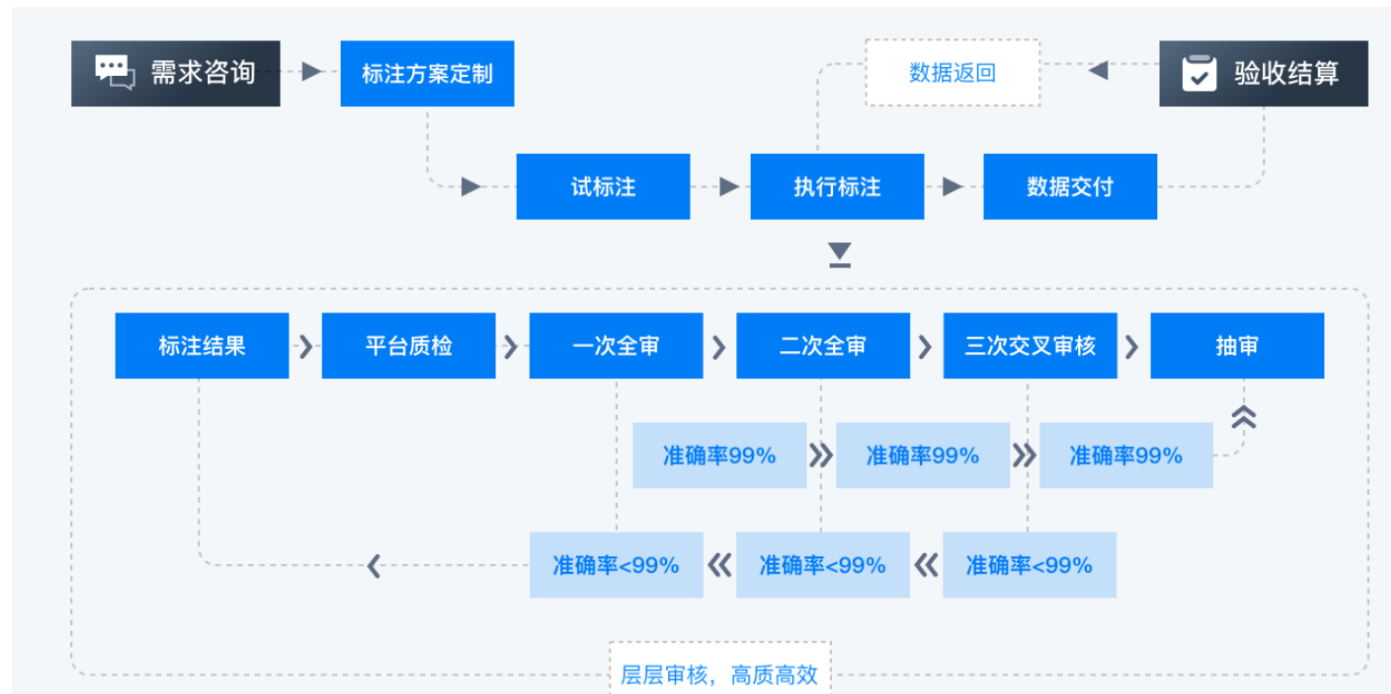
**04** 迁移学习前沿

为人师表

# 1.迁移学习背景



有多少**人工**，就有多少智能！



数据的标注是一个**耗时**且**昂贵**的操作！

# 1.迁移学习背景



TABLE 3: Compare AlexNet training with different approaches.

|                         | Batch Size | Processor         | GPU Interconnect | Time            | Top-1 Accuracy |
|-------------------------|------------|-------------------|------------------|-----------------|----------------|
| You et al. [41]         | 512        | DGX-1 station     | NVLink           | 6 hours 10 mins | 58.8%          |
| You et al. [41]         | 32K        | CPU x 1024        | -                | 11 mins         | 58.6%          |
| Jia et al. [18]         | 64K        | Pascal GPU x 512  | 100 Gbps         | 5 mins          | 58.8%          |
| Jia et al. [18]         | 64K        | Pascal GPU x 1024 | 100 Gbps         | 4 mins          | 58.7%          |
| This Work (DenseCommu)  | 64K        | Volta GPU x 512   | 56 Gbps          | 2.6 mins        | 58.7%          |
| This Work (SparseCommu) | 64K        | Volta GPU x 512   | 56 Gbps          | 1.5 mins        | 58.2%          |

TABLE 4: Compare ResNet-50 training with different approaches.

|                        | Batch Size | Processor         | GPU Interconnect | Time     | Top-1 Accuracy |
|------------------------|------------|-------------------|------------------|----------|----------------|
| Goyal et al. [3]       | 8K         | Pascal GPU x 256  | 56 Gbps          | 1 hour   | 76.3%          |
| Smith et al. [42]      | 16K        | Full TPU Pod      | -                | 30 mins  | 76.1%          |
| Codreanu et al. [43]   | 32K        | KNL x 1024        | -                | 42 mins  | 75.3%          |
| You et al. [41]        | 32K        | KNL x 2048        | -                | 20 mins  | 75.4%          |
| Akiba et al. [38]      | 32K        | Pascal GPU x 1024 | 56 Gbps          | 15 mins  | 74.9%          |
| Jia et al. [18]        | 64K        | Pascal GPU x 1024 | 100 Gbps         | 8.7 mins | 76.2%          |
| Jia et al. [18]        | 64K        | Pascal GPU x 2048 | 100 Gbps         | 6.6 mins | 75.8%          |
| Mikami et al. [44]     | 68K        | Volta GPU x 2176  | 200 Gbps         | 3.7 mins | 75.0%          |
| This Work (DenseCommu) | 64K        | Volta GPU x 512   | 56 Gbps          | 7.3 mins | 75.3%          |

Sun P, Feng W, Han R, et al. Optimizing network performance for distributed dnn training on gpu cluster  
Imagenet/alexnet training in 1.5 minutes[J]. arXiv preprint arXiv:1902.06855, 2019.

| Solver   | batch size | steps | F1 score on dev set | TPUs | Time   |
|----------|------------|-------|---------------------|------|--------|
| Baseline | 512        | 1000k | 90.395              | 16   | 81.4h  |
| LAMB     | 512        | 1000k | 91.752              | 16   | 82.8h  |
| LAMB     | 1k         | 500k  | 91.761              | 32   | 43.2h  |
| LAMB     | 2k         | 250k  | 91.946              | 64   | 21.4h  |
| LAMB     | 4k         | 125k  | 91.137              | 128  | 693.6m |
| LAMB     | 8k         | 62500 | 91.263              | 256  | 390.5m |
| LAMB     | 16k        | 31250 | 91.345              | 512  | 200.0m |
| LAMB     | 32k        | 15625 | 91.475              | 1024 | 101.2m |
| LAMB     | 64k/32k    | 8599  | 90.584              | 1024 | 76.19m |

You Y, Li J, Reddi S, et al. Large batch optimization for deep learning: Training bert in 76 minutes[J]. arXiv preprint arXiv:1904.00962, 2019.

大数据需要强计算能力的设备来进行存储和计算。近些年,深度学习的**数据量**和**模型大小**在迅速增加,高昂的计算成本是普通个人用户无法长期负担的

GPU云服务器gn6v

最高配置8张NVIDIA, 16G显存V100  
计算卡, 336G DDR4内存

8核32G1个月

V100GPU1M带宽40G高效云盘

包月6.1折1年5折3年4.2折

¥4648.20/月起

¥2971.80/月

GPU云服务器gn5

最高配置8张NVIDIA 16G显存P100计  
算卡, 高性能NVMe SSD本地盘

32核240G1个月

P100GPU16GB单块显存SSD本地盘

包月9.5折1年7.5折2年6折

¥17842.00/月起

GPU云服务器gn5i

最高配置2张NVIDIA 8G显存P4计算  
卡, 224G DDR4内存

16核64G1个月

P4GPU8GB单块显存

包月9.5折1年7.5折2年6折

¥4275.00/月起

¥225.00/月

GPU云服务器gn6i

最高配置4张NVIDIA 16G显存T4计算  
卡, 372G DDR4内存

16核62G1个月

T4GPU16GB单块显存

包月9.5折1年7.5折2年6折

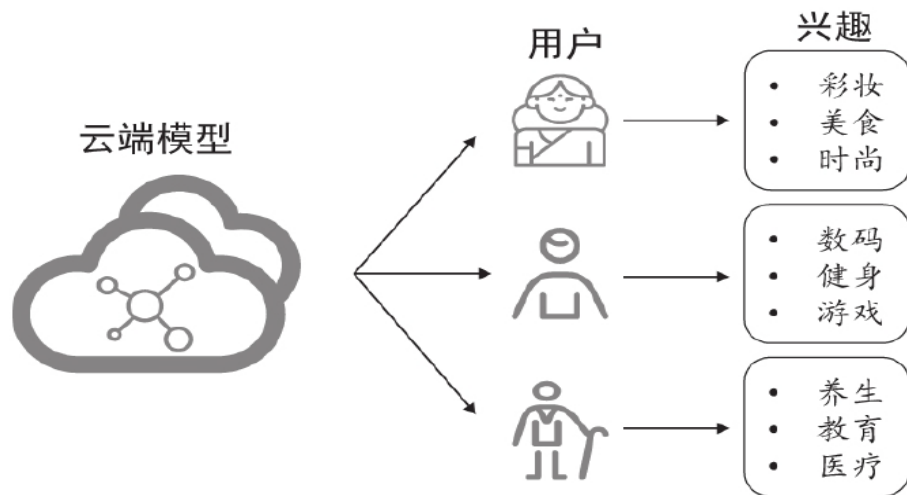
¥4836.00/月起

迁移学习提供了一种基于大数据“预训练”的模型在自己的特定数据集上进行“微调”的技术,普通人也能利用这些数据和模型

# 1.迁移学习背景



## 普适化模型→个性化需求



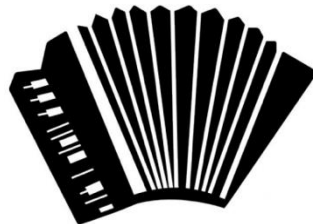
利用迁移学习的思想，我们可以将那些大公司在大数据上**训练好的模型**，迁移到我们的任务中，针对我们的任务进行**微调**，从而也可以拥有在大数据上训练好的模型。更进一步，可以将这些模型针对具体任务进行自适应更新，取得更好的效果。



## 2. 迁移学习概念

□ 概念：指利用数据、任务、或模型之间的相似性，将在旧领域学习过的模型，应用于新领域的一种学习过程。

- 球类运动相似：乒乓球高手可很快学会打网球
- 乐器相通：手风琴和钢琴也有相通的地方



应用领域



语义匮乏条件下不同语言的相互翻译学习



不同视角、不同背景、不同光照的图像识别



不同用户、不同设备、不同位置的识别



不同领域、不同背景下的文本翻译、舆情分析



不同用户、不同接口、不同情境下的人机交互

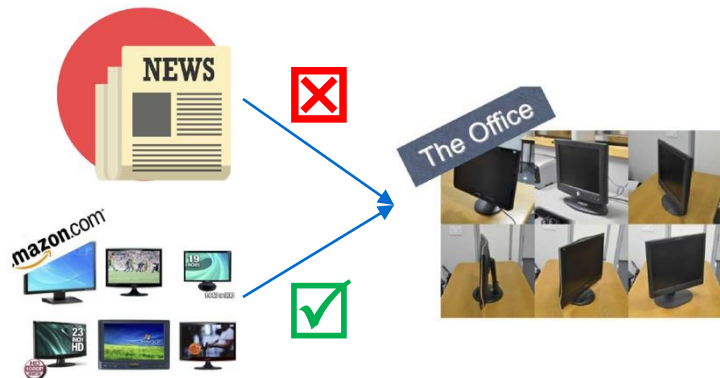


不同场景、不同时间、不同设备的室内定位

## 2. 迁移学习概念

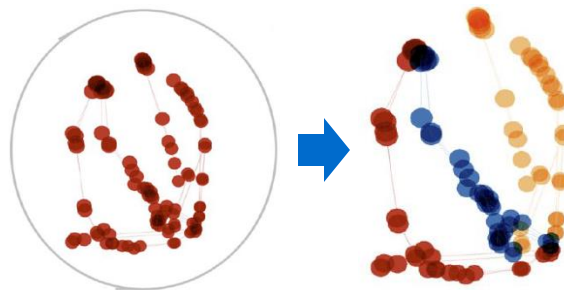
### □ 迁移学习的三个基本问题

#### ● 何时迁移



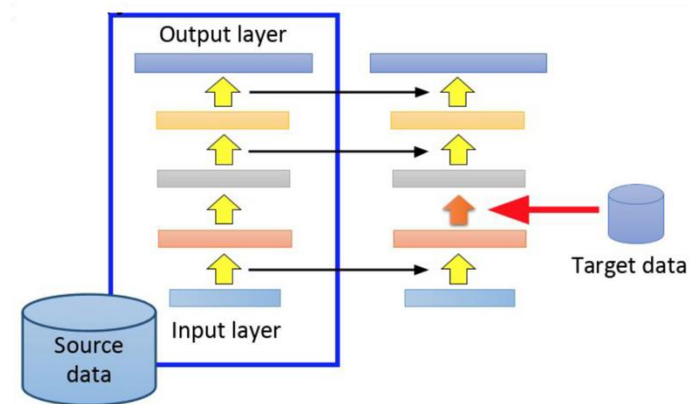
原问题和目标问题具有相关性

#### ● 何处迁移



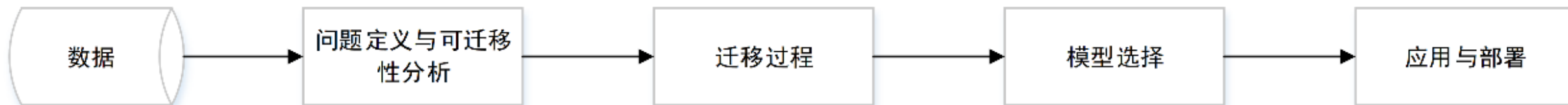
要迁移的知识往往来源于相似的模式或特征

#### ● 如何迁移



建立原问题和目标问题的关联

### □ 迁移学习的一般过程



## 2. 迁移学习概念

□ 迁移学习的重要概念：

**领域：** 是机器学习的主体，主要由两个部分构成：**数据和生成这些数据的概率分布。**

通常用 $D$ 表示一个领域，可表示为 $D = \{X, Y, P(x, y)\}$ 。其中 $X, Y$ 分别为数据所处的特征空间和标签空间， $P$ 为数据服从的概率分布。





## 2. 迁移学习概念

$$\text{领域 } D = \{X, Y, P(x, y)\}$$

### 域不同的三种情形:

#### ➤ 特征空间不同, 即 $X_s \neq X_t$ :

例如, 当源域为RGB彩色图像、目标域为黑白二值图像

源域样本:



目标域样本:



#### ➤ 标签空间不同, 即 $Y_s \neq Y_t$ :

例如, 在分类问题中, 源域和目标域类别不完全相同

源域只有猫和狗两种类别:



猫

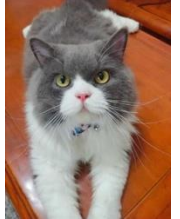


狗

目标域除了猫和狗外还包含品种



金毛犬

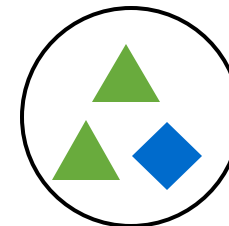


英国短毛猫

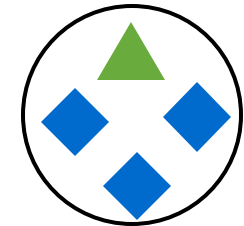
#### ➤ 概率分布不同, 即 $P_s(x, y) \neq P_t(x, y)$ : (重点介绍)

特指即使两个领域的特征空间和类别空间都相同, 其联合概率分布也会存在不匹配的问题

源域:



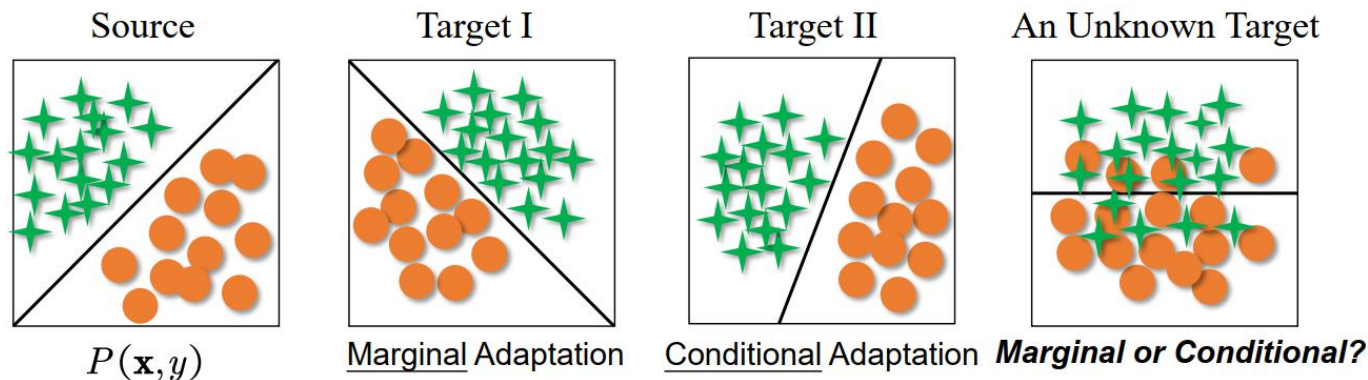
目标域:



## 2. 迁移学习概念



- $P_s(x, y) \neq P_t(x, y)$  说明了什么?
  - 贝叶斯定理:  $P(x, y) = P(y|x) P(x)$
  - 若  $P_s(y|x) = P_t(y|x)$ , 且边缘分布不同:
    - $P_s(x) \neq P_t(x)$ , 采用边缘分布自适应方法
  - 若  $P_s(x) = P_t(x)$ , 且条件分布不同:
    - $P_s(y|x) \neq P_t(y|x)$ , 采用条件分布自适应方法



(Yu et al. Transfer learning with dynamic adversarial adaptation network. ICDM 2019.)

## 2.迁移学习概念



### □迁移学习的核心思路：

- 找到源域和目标域之间的差异性，并将整体工作归结为两方面

- 一是度量两个领域的相似性，不仅定性地告诉我们它们是否相似，更定量地给出相似程度
- 二是以度量为准则，通过我们所要采用的学习手段，增大两个领域之间的相似性，完成迁移学习。

### □ 迁移学习的统一数学表征:

- 从机器学习的最小化经验风险准则 (ERM) 出发

$$f^* = \arg \min_f \frac{1}{m} \sum_{i=1}^m L(f(x_i), y_i), \quad L(\cdot, \cdot) \text{ 是损失函数}$$

- 使用一些方法来减少域差异, 得到迁移学习的统一表征

$$f^* = \arg \min_{f \in \mathcal{H}} \frac{1}{N_s} \sum_{i=1}^{N_s} L(v_i f(x_i), y_i) + \lambda R(T(D_s), T(D_t))$$

➤ 其中:

- $v \in \mathbb{R}^{N_s}, \quad v_i \in [0, 1]$ 。  $N_s$  为源域样本的数量
- $T$  为作用于源域和目标域上的特征变换函数
- $R(T(D_s), T(D_t))$  为迁移正则化项, 用于定量衡量源域和目标域之间的分布距离

### 3. 迁移学习方法



#### □ 迁移学习方法分类:

- 三种基本的迁移学习类型

$$f^* = \arg \min_{f \in \mathcal{H}} \frac{1}{N_s} \sum_{i=1}^{N_s} L(v_i f(x_i), y_i) + \lambda R(T(D_s), T(D_t))$$

#### 基于样本的迁移学习

- 目标为根据源域和目标域的相似度来学习源域样本的权重  $v_i$

#### 基于特征的迁移学习

- 目标为学习一个特征变换  $T$  来减小迁移正则化项  $R(\cdot, \cdot)$

#### 基于模型的迁移学习

- 目标为学习如何将源域的判别函数  $f(\cdot, \cdot)$  对目标域数据进行正则化和微调



### 3. 迁移学习方法



#### □ 基于样本的迁移方法:

- 核心思想: 增大源域样本中与目标域样本差异度小的比重



根据  $v_i$  取值范围划分

- 样本选择法:  $v_i \in \{0,1\}$
- 权重自适应法:  $v_i \in [0,1]$

### 3.迁移学习方法

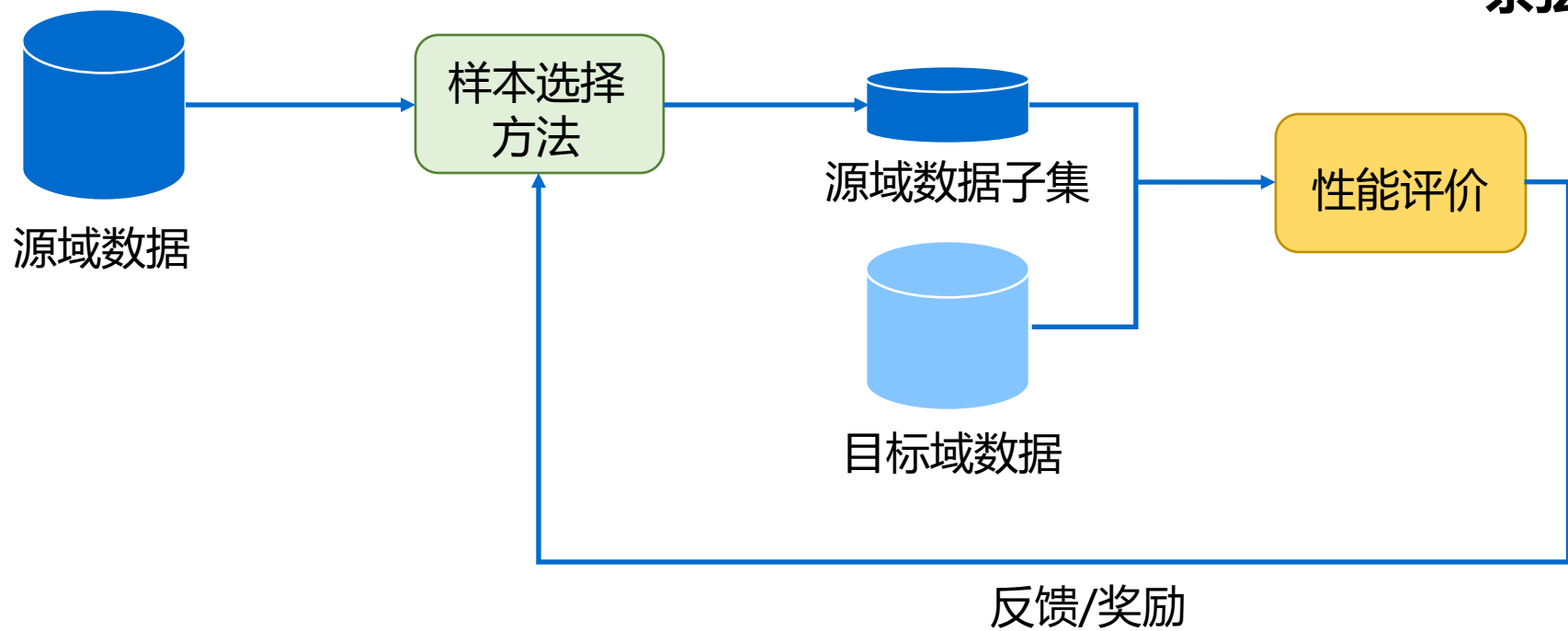


#### □基于样本的迁移方法---样本选择

##### • 基本框架

样本筛选方法:

- KL散度
- 交叉熵
- 网络损失
- 余弦相似度



### 3. 迁移学习方法



#### □ 基于样本的迁移方法---样本权重自适应

- 基本思路，原始损失函数

$$f^* = \arg \min_f \frac{1}{m} \sum_{i=1}^m L(f(x_i), y_i)$$

- 乘以density ratio项  $\beta = \frac{P_t(x)}{P_s(x)}$  调节损失权重

$$f^* = \arg \min_f \frac{1}{m} \sum_{i=1}^m \beta_i L(f(x_i), y_i)$$

- 根据源域和目标域样本自动计算  
(重合度约高约接近于1)

### 3.迁移学习方法



#### □基于样本的迁移方法---样本权重自适应

##### • 相关算法

- 逻辑回归

$$\min_{\theta} \sum_{i=1}^m -\beta_i \log P(y_i|x_i, \theta) + \frac{\lambda}{2} ||\theta||^2$$

- 支持向量机Support vector machine

$$\min_{\theta, \xi} \frac{1}{2} ||\theta||^2 + C \sum_{i=1}^m -\beta_i \xi_i$$

$$s. t. \langle \Phi(x_i, y_i) - \Phi(x_i, y), \theta \rangle \geq 1 - \frac{\xi_i}{\Delta(y_i, y)}, \xi_i > 0$$

- 深度学习等

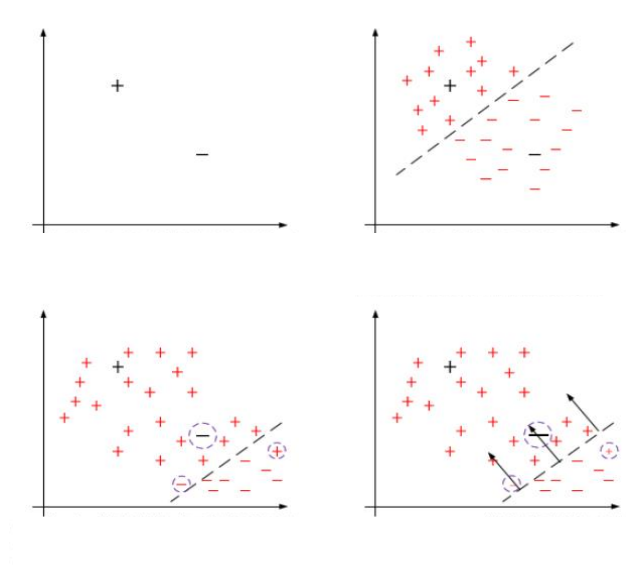
### 3. 迁移学习方法



#### □ 基于样本的迁移方法---样本损失调节

**Tradaboost: Transfer + Adaboost**

**训练思路：将分类器分的不好的样本的权重降低---与Adaboost相反**



[1] Dai et al. Boosting for transfer learning. ICML 2007. Figures are from the authors thesis.

### 3.迁移学习方法



#### □基于特征的迁移方法：

- 将源域和目标域的差异性视为两个域内样本数据的概率分布的差异性进行研究，根据采用显式或隐式的距离度量分为两类
  - 显式度量：由预定义好的距离公式产生的度量，具有特定形式，常用的距离度量有欧式距离、闵可夫斯基距离、马氏距离、余弦相似度等
  - 隐式度量：并非预先定义好的，而是可以在数据中动态学习的、更适合数据分布的度量



### 3. 迁移学习方法



#### □ 基于特征的迁移方法：

- 迁移学习中使用最广泛的距离度量之一 —— 最大均值差异 (Maximum Mean Discrepancy, MMD)

$$\text{MMD}^2(A, B) = \left\| \sum_{i=1}^{n1} \phi(a_i) - \sum_{j=1}^{n2} \phi(b_j) \right\|^2$$

$A, B$ : 两个样本集合

$a_i$ : 集合 $A$ 中的样本

$b_j$ : 集合 $B$ 中的样本

$\phi$ : 特征映射函数

$\phi(b_j)$ : 样本 $b_j$ 的特征

$\phi(a_i)$ : 样本 $a_i$ 的特征

直观解释：计算两个集合中样本的特征变换后分布之前的距离

#### □ 基于特征的迁移方法:

- 迁移学习中使用最广泛的距离度量之一 —— 最大均值差异 (Maximum Mean Discrepancy, MMD)

- 给定  $x \sim P, y \sim Q$ ,  $f$  对应一个特征图:  $x \rightarrow \mathcal{H}$ ,  $\mathcal{H}$  为再生希尔伯特空间, 可得

$$MMD(P, Q, \mathcal{F}) := \sup_{f \in \mathcal{F}} \mathbb{E}_P[f(x)] - \mathbb{E}_Q[f(y)]$$

- 经验估计

$$MMD(P, Q, \mathcal{F}) := \sup_{f \in \mathcal{F}} \mathbb{E}_P \left[ \frac{1}{m} \sum_{i=1}^m f(x_i) - \frac{1}{n} \sum_{j=1}^n f(y_j) \right]$$

### 3.迁移学习方法



#### □基于特征的迁移方法:

##### ➤ 基于特征的迁移方法优化目标:

$$f^* = \arg \min_{f \in \mathcal{H}} \frac{1}{N_s} \sum_{i=1}^{N_s} L(f(x_i), y_i) + \lambda R(T(D_s), T(D_t)) \text{ 这里 } R(\cdot, \cdot) \text{ 即为选择的距离度量MMD}$$

##### ➤ 迁移学习中的概率分布差异方法与假设:

| 方法      | 假设                         | 优化目标                                                                    |
|---------|----------------------------|-------------------------------------------------------------------------|
| 边缘分布自适应 | $P_s(y x) = P_t(y x)$      | $\min MMD(P_s(x), P_t(x), f)$                                           |
| 条件分布自适应 | $P_s(x) = P_t(x)$          | $\min MMD(P_s(y x), P_t(y x), f)$                                       |
| 联合分布自适应 | $P_s(x, y) \neq P_t(x, y)$ | $\min MMD(P_s(x), P_t(x), f) + MMD(P_s(y x), P_t(y x), f)$              |
| 动态分布自适应 | $P_s(x, y) \neq P_t(x, y)$ | $\min(1 - \mu) MMD(P_s(x), P_t(x), f) + \mu MMD(P_s(y x), P_t(y x), f)$ |

( $f$  为特征变换函数)

第四种为最一般的形式, 前三种为第四种取特值的退化形式!

### 3.迁移学习方法



#### □动态分布自适应问题:

- 分布自适应因子 $\mu$ 的计算方法

- 随机猜测法: 任意从 $[0, 1]$ 区间内选择一个 $\mu$ 的值, 然后进行动态迁移学习。重复此过程 $t$ 次, 则随机猜测法最终的迁移结果为 $t$ 次迁移学习结果的均值。
- 最大最小平均法: 在 $[0, 1]$ 区间内, 以固定步长取等距的 $\mu$ , 多样重复学习多次, 最终的迁移结果为多次迁移学习的均值。



**有没有一种精确计算且具有理论依据的计算方法?**

### 3. 迁移学习方法



#### □ 动态分布自适应问题:

##### • 分布自适应因子 $\mu$ 的计算方法

利用领域的整体和局部性质来定量计算  $\longrightarrow$  采用  $\mathcal{A} - distance$  作为基本度量方式

$$d_A(D_s, D_t) = 2(1 - \underline{2\epsilon(h)})$$

$\epsilon(h)$ : 线性分类器区分源域 $D_s$ 和目标域 $D_t$ 的误差

$$\hat{\mu} = 1 - \frac{d_M}{\underline{d_M} + \sum_{c=1}^C \underline{d_c}}$$

$d_M$  为边缘分布距离

$$d_M = d_A(D_s, D_t)$$

$d_c$  为表示对应于类别 $c$ 的条件分布距离

$$d_c = d_A(D_s^{(c)}, D_t^{(c)})$$

Wang J, Feng W, Chen Y, et al. Visual domain adaptation with manifold embedded distribution alignment[C]//Proceedings of the 26th ACM international conference on Multimedia. 2018: 402-410.

### 3. 迁移学习方法



#### □ 动态分布自适应问题:

- 动态分布自适应问题求解:

回顾优化目标, 由边缘分布距离和条件分布距离加权计算:

$$\min(1 - \mu) \underbrace{MMD(P_s(x), P_t(x), f)}_{\text{边缘分布距离}} + \mu \underbrace{MMD(P_s(y|x), P_t(y|x), f)}_{\text{条件分布距离}}$$

边缘分布距离

条件分布距离

经过特征变换后边缘分布距离:

$$MMD(P_s(x), P_t(x), f) = \left\| \frac{1}{N_s} \sum_{i=1}^{N_s} A^T x_i - \frac{1}{N_t} \sum_{j=1}^{N_t} A^T x_j \right\|_{\mathcal{H}}^2$$

#### □ 动态分布自适应问题:

- 动态分布自适应问题求解:

由于目标域样本存在少标签和无标签的情况, 直接求解条件分布距离很困难

可以换个角度, 利用类条件概率 $P_t(x|y)$ 。根据贝叶斯公式 $P_t(y|x) = \frac{P_t(x|y)P_t(y)}{P_t(x)}$ , 如果忽略 $P_t(y)$ , 则可用类条件概率近似估计条件概率

$$MMD(P_s(y|x), P_t(y|x), f) \approx MMD(P_s(x|y), P_t(x|y), f)$$

经过特征变换后条件分布距离:

$$MMD(P_s(x|y), P_t(x|y), f) = \sum_{c=1}^C \left\| \frac{1}{N_s^c} \sum_{x_i \in D_s^{(c)}} A^T x_i - \frac{1}{N_t^c} \sum_{x_i \in D_t^{(c)}} A^T x_i \right\|_{\mathcal{H}}^2$$



### 3. 迁移学习方法



#### □ 动态分布自适应问题:

- 动态分布自适应问题求解:

用MMD进行迁移学习的最终表达形式为:

$$\text{tr}(A^T X M X^T A)$$

$M$ 为MMD矩阵, 计算式为:  $M = (1 - \mu)M_0 + \mu \sum_{c=1}^C M_c$

$$(M_0)_{ij} = \begin{cases} \frac{1}{N_s^2}, & x_i, x_j \in D_s \\ \frac{1}{N_t^2}, & x_i, x_j \in D_t \\ -\frac{1}{N_s N_t}, & \text{otherwise} \end{cases} \quad (M_c)_{ij} = \begin{cases} \frac{1}{(N_s^{(c)})^2}, & x_i, x_j \in D_s^{(c)} \\ \frac{1}{(N_t^{(c)})^2}, & x_i, x_j \in D_t^{(c)} \\ -\frac{1}{N_s^{(c)} N_t^{(c)}}, & \begin{cases} x_i \in D_s^{(c)}, x_j \in D_t^{(c)} \\ x_i \in D_t^{(c)}, x_j \in D_s^{(c)} \end{cases} \\ 0, & \text{otherwise} \end{cases}$$

如何得到 $P(x|y)$ ?  
伪表现. 迭代!

推导过程: [MMD计算的核技巧公式推导 - 知乎 \(zhihu.com\)](https://zhuanlan.zhihu.com/p/100000000)

## 3.迁移学习方法



### □动态分布自适应问题:

#### • 动态分布自适应问题伪代码:

计算条件概率距离时如何获取目标域中样本的类别信息? ———> 采用伪标签并进行迭代

- step1.选取一个分类器算法使用源域数据训练并为目标域中的样本打上伪标签。
- step2.计算分布自适应因子 $\mu$
- step3.计算特征提取矩阵A
- step4.使用特征提取矩阵A对原数据提取成分，再输入分类器重新分类，打上新标签进行标签更新
- step5.使用新标签，回到第2步，不断迭代直到收敛

### 3.迁移学习方法



#### □动态分布自适应问题:

##### • 动态分布自适应问题求解:

$$\begin{aligned} \bullet \text{Distance}(D_s, D_t) &\approx \text{MMD}(P_s(x), P_t(x), f) \\ &= \sup_{f \in \mathcal{F}} \mathbb{E}_P \left[ \frac{1}{m} \sum_{i=1}^m f(x_i) - \frac{1}{n} \sum_{j=1}^n f(y_j) \right] \end{aligned}$$

$$\begin{aligned} \bullet (\text{线性映射形式}) \quad &= \text{tr}(\mathbf{A}^T \mathbf{X} \mathbf{M} \mathbf{X}^T \mathbf{A}) \\ &\mathbf{X} = [\mathbf{X}_s, \mathbf{X}_t] \in \mathbb{R}^{d \times (m+n)}, \mathbf{A} \in \mathbb{R}^{(m+n) \times (m+n)} \end{aligned}$$

$$\bullet (\text{核函数形式}) = \text{tr}(\mathbf{K} \mathbf{M})$$

$$K = \begin{bmatrix} K_{s,s} & K_{s,t} \\ K_{t,s} & K_{t,t} \end{bmatrix} \quad M_{i,j} = \begin{cases} \frac{1}{m^2}, x_i, x_j \in D_s \\ \frac{1}{n^2}, x_i, x_j \in D_t \\ -\frac{1}{mn}, \text{otherwise} \end{cases}$$

$$\begin{aligned} &\text{Transfer Component Analysis (TCA)}^{[1]} \\ &\min \text{tr}(\mathbf{K} \mathbf{M}) - \lambda \text{tr}(\mathbf{K}) \end{aligned}$$

Pan S J, Tsang I W, Kwok J T, et al. Domain adaptation via transfer component analysis[J]. IEEE transactions on neural networks, 2010, 22(2): 199-210.

### 3. 迁移学习方法



#### □ 动态分布自适应问题:

##### • 动态分布自适应问题求解:

Use kernel trick:

$$\begin{aligned} & \left\| \frac{1}{n_s} \sum_{i=1}^{n_s} \phi(\mathbf{x}_i) - \frac{1}{n_t} \sum_{j=1}^{n_t} \phi(\mathbf{x}_j) \right\|^2 \\ &= \text{tr} \left( \begin{bmatrix} \phi(\mathbf{x}_s) & \phi(\mathbf{x}_t) \end{bmatrix} \begin{bmatrix} \frac{1}{n_s^2} \mathbf{1}\mathbf{1}^T & \frac{-1}{n_s n_t} \mathbf{1}\mathbf{1}^T \\ \frac{-1}{n_s n_t} \mathbf{1}\mathbf{1}^T & \frac{1}{n_t^2} \mathbf{1}\mathbf{1}^T \end{bmatrix} \begin{bmatrix} \phi(\mathbf{x}_s)^T \\ \phi(\mathbf{x}_t)^T \end{bmatrix} \right) \\ &= \text{tr} \left( \begin{bmatrix} \phi(\mathbf{x}_s)^T \\ \phi(\mathbf{x}_t)^T \end{bmatrix} \begin{bmatrix} \phi(\mathbf{x}_s) & \phi(\mathbf{x}_t) \end{bmatrix} \begin{bmatrix} \frac{1}{n_s^2} \mathbf{1}\mathbf{1}^T & \frac{-1}{n_s n_t} \mathbf{1}\mathbf{1}^T \\ \frac{-1}{n_s n_t} \mathbf{1}\mathbf{1}^T & \frac{1}{n_t^2} \mathbf{1}\mathbf{1}^T \end{bmatrix} \right) \\ &= \text{tr} \left( \begin{bmatrix} \langle \phi(\mathbf{x}_s), \phi(\mathbf{x}_s) \rangle & \langle \phi(\mathbf{x}_s), \phi(\mathbf{x}_t) \rangle \\ \langle \phi(\mathbf{x}_t), \phi(\mathbf{x}_s) \rangle & \langle \phi(\mathbf{x}_t), \phi(\mathbf{x}_t) \rangle \end{bmatrix} \mathbf{M} \right) \\ &= \text{tr} \left( \begin{bmatrix} K_{s,s} & K_{s,t} \\ K_{t,s} & K_{t,t} \end{bmatrix} \mathbf{M} \right) \end{aligned}$$

where

$$(M)_{ij} = \begin{cases} \frac{1}{n_s n_s}, & \mathbf{x}_i, \mathbf{x}_j \in \mathcal{D}_s \\ \frac{1}{n_t n_t}, & \mathbf{x}_i, \mathbf{x}_j \in \mathcal{D}_t \\ \frac{-1}{n_s n_t}, & \text{otherwise} \end{cases}$$

No kernel trick:

$$\begin{aligned} & \left\| \frac{1}{n_s} \sum_{i=1}^{n_s} \mathbf{A}^T \mathbf{x}_i - \frac{1}{n_t} \sum_{j=n_s+1}^{n_s+n_t} \mathbf{A}^T \mathbf{x}_j \right\|^2 \\ &= \left\| \frac{1}{n_s} \mathbf{A}^T \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_{n_s} \end{bmatrix}_{1 \times n_s} \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}_{n_s \times 1} - \frac{1}{n_t} \mathbf{A}^T \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_{n_t} \end{bmatrix}_{1 \times n_t} \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}_{n_t \times 1} \right\|^2 \\ &= \text{tr} \left( \frac{1}{n_s^2} \mathbf{A}^T \mathbf{X}_s \mathbf{1} (\mathbf{A}^T \mathbf{X}_s \mathbf{1})^T + \frac{1}{n_t^2} \mathbf{A}^T \mathbf{X}_t \mathbf{1} (\mathbf{A}^T \mathbf{X}_t \mathbf{1})^T - \frac{1}{n_s n_t} \mathbf{A}^T \mathbf{X}_s \mathbf{1} (\mathbf{A}^T \mathbf{X}_t \mathbf{1})^T - \frac{1}{n_s n_t} \mathbf{A}^T \mathbf{X}_t \mathbf{1} (\mathbf{A}^T \mathbf{X}_s \mathbf{1})^T \right) \\ &= \text{tr} \left( \frac{1}{n_s^2} \mathbf{A}^T \mathbf{X}_s \mathbf{1} \mathbf{1}^T \mathbf{X}_s^T \mathbf{A} + \frac{1}{n_t^2} \mathbf{A}^T \mathbf{X}_t \mathbf{1} \mathbf{1}^T \mathbf{X}_t^T \mathbf{A} - \frac{1}{n_s n_t} \mathbf{A}^T \mathbf{X}_s \mathbf{1} \mathbf{1}^T \mathbf{X}_t^T \mathbf{A} - \frac{1}{n_s n_t} \mathbf{A}^T \mathbf{X}_t \mathbf{1} \mathbf{1}^T \mathbf{X}_s^T \mathbf{A} \right) \\ &= \text{tr} \left[ \mathbf{A}^T \left( \frac{1}{n_s^2} \mathbf{1}\mathbf{1}^T \mathbf{X}_s^T \mathbf{X}_s + \frac{1}{n_t^2} \mathbf{1}\mathbf{1}^T \mathbf{X}_t^T \mathbf{X}_t - \frac{1}{n_s n_t} \mathbf{1}\mathbf{1}^T \mathbf{X}_s^T \mathbf{X}_t - \frac{1}{n_s n_t} \mathbf{1}\mathbf{1}^T \mathbf{X}_t^T \mathbf{X}_s \right) \mathbf{A} \right] \\ &= \text{tr} \left( \mathbf{A}^T \begin{bmatrix} \mathbf{X}_s & \mathbf{X}_t \end{bmatrix} \begin{bmatrix} \frac{1}{n_s^2} \mathbf{1}\mathbf{1}^T & \frac{-1}{n_s n_t} \mathbf{1}\mathbf{1}^T \\ \frac{-1}{n_s n_t} \mathbf{1}\mathbf{1}^T & \frac{1}{n_t^2} \mathbf{1}\mathbf{1}^T \end{bmatrix} \begin{bmatrix} \mathbf{X}_s \\ \mathbf{X}_t \end{bmatrix} \mathbf{A} \right) \\ &= \text{tr} (\mathbf{A}^T \mathbf{X} \mathbf{M} \mathbf{X}^T \mathbf{A}) \end{aligned}$$

推导过程: [MMD计算的核技巧公式推导 - 知乎 \(zhihu.com\)](https://zhuanlan.zhihu.com/p/100000000)

### 3.迁移学习方法



#### □动态分布自适应问题:

- 动态分布自适应问题求解:

优化目标为最小化MMD距离:

$$\min(\text{tr}(A^T X M X^T A))$$

加上方差最大化约束:

$$\max(\text{tr}(A^T X H X^T A))$$

得到最后的优化目标:

$$\min \left( \frac{\text{tr}(A^T X M X^T A)}{\text{tr}(A^T X H X^T A)} \right)$$

### 3.迁移学习方法



#### □动态分布自适应问题:

- 动态分布自适应问题求解:

根据瑞利商对原有约束问题进行变换:

$$\min(\text{tr}(A^T X M X^T A)) + \lambda \|A\|_F^2$$

$$s. t. \quad A^T X H X^T A = I$$

用拉格朗日法可对含约束条件的最小化问题进行求解:

$$(X M X^T + \lambda I) = X H X^T A \Phi$$

$\lambda$ : 正则化系数

$\Phi$ : 拉格朗日乘子, 在Matlab和Python等编程语言中可以直接求解

完整求解过程见《迁移学习导论第二版》(王晋东、陈益强)

### 3. 迁移学习方法

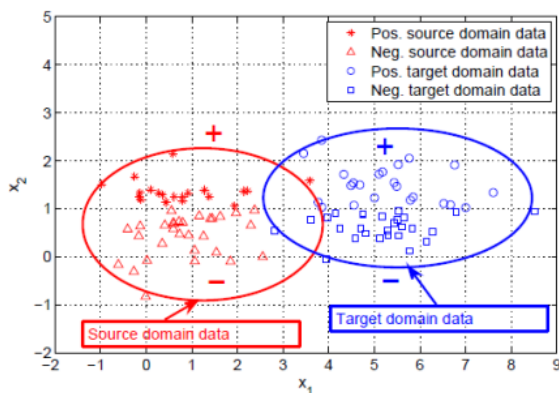
$$\min(1 - \mu) MMD(P_S(x), P_t(x), f) + \mu MMD(P_S(y|x), P_t(y|x), f)$$



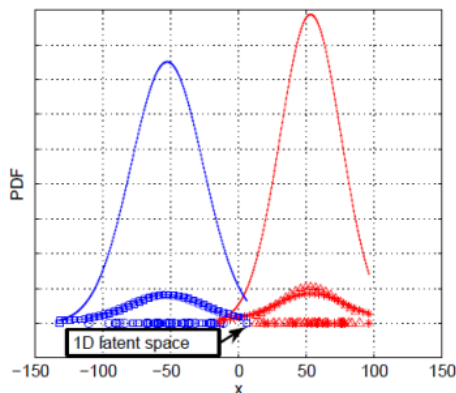
#### □ 动态分布自适应问题:

- $\mu = 0$  TCA (Transfer Component Analysis)

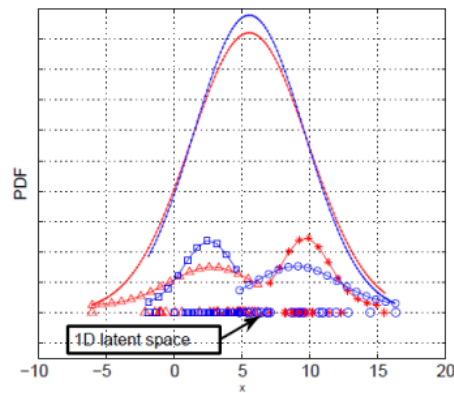
$$\min \text{tr}(\mathbf{KM}) - \lambda \text{tr}(\mathbf{K})$$



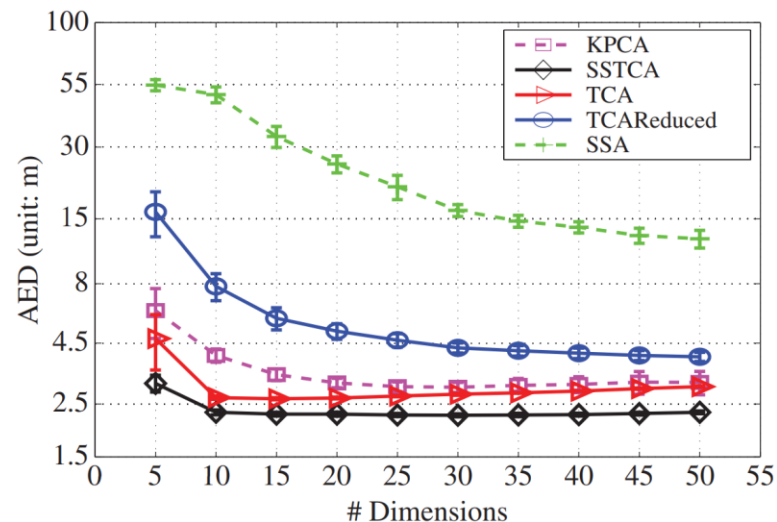
Original feature space



PCA



TCA



### 3. 迁移学习方法

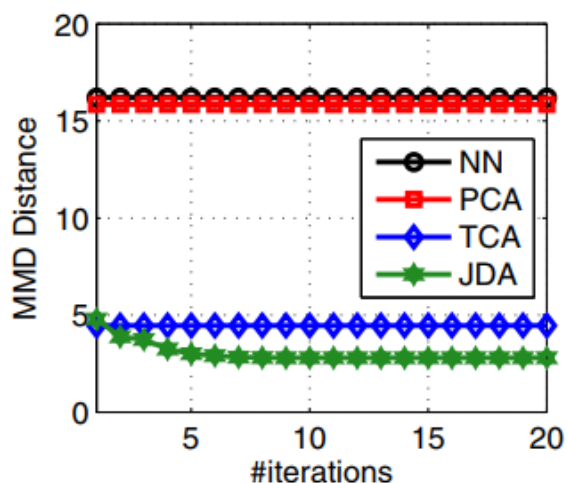
$$\min(1 - \mu) MMD(P_S(x), P_t(x), f) + \mu MMD(P_S(y|x), P_t(y|x), f)$$



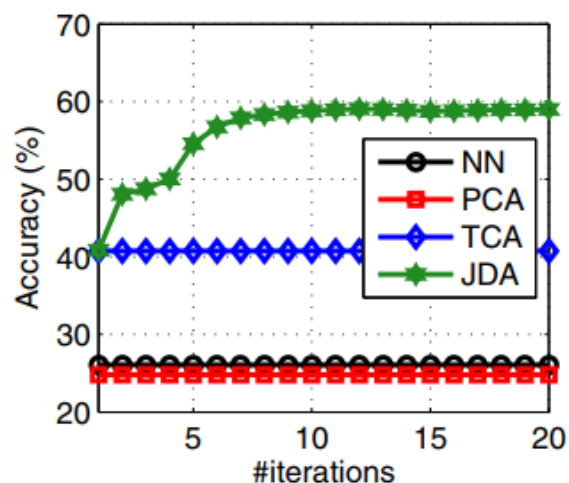
#### □ 动态分布自适应问题:

- $\mu = 0.5$  JDA (Joint distribution adaptation)

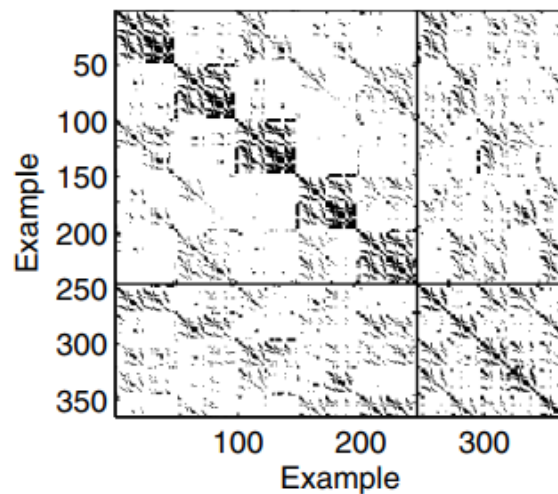
$$\min \sum_{c=0}^C \text{tr}(\mathbf{A}^T \mathbf{X} \mathbf{M}_c \mathbf{X}^T \mathbf{A}) + \lambda \|\mathbf{A}\|^2$$



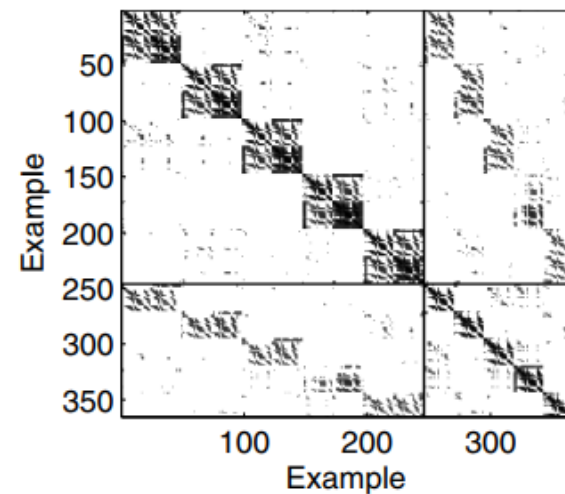
(a) MMD distance w.r.t. #iterations



(b) Accuracy (%) w.r.t. #iterations



(c) Similarity of TCA embedding



(d) Similarity of JDA embedding



### 3. 迁移学习方法

$$\min(1 - \mu) MMD(P_S(x), P_t(x), f) + \mu MMD(P_S(y|x), P_t(y|x), f)$$



#### □ 动态分布自适应问题:

- 基于深度学习的迁移学习

**DDC: Deep domain confusion**

**DDAN: Dynamic distribution adaptation network**

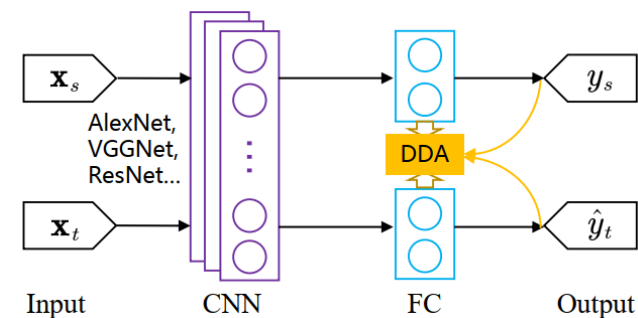
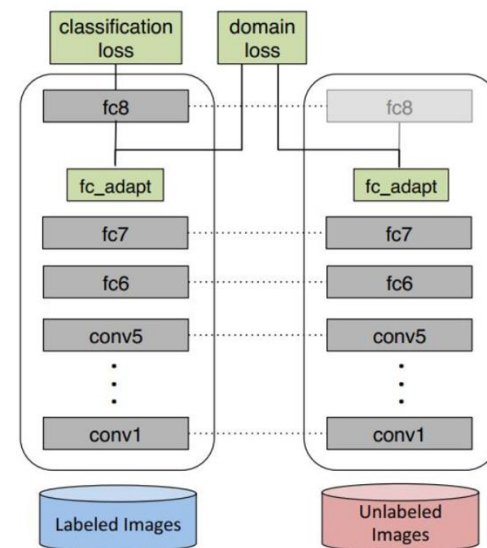
Network loss:  $L = L_c(x_i, y_i) + \lambda \cdot Distance(D_s, D_t)$

- $L_c(\cdot, \cdot)$ : Classification / regression loss, e.g. cross-entropy
- $Distance(\cdot, \cdot)$ : MMD distance, e.g. TCA, JDA, DDA
- Gradient:  $\nabla_{\Theta} = \frac{\partial L_c(x_i, y_i)}{\partial \Theta} + \lambda \frac{\partial Distance(D_s, D_t)}{\partial \Theta}$
- End-to-end Training

[1] Tzeng et al. Deep domain confusion: maximizing domain confusion. arXiv 2014.

[2] Wang et al. Transfer learning with dynamic distribution adaptation. ACM TIST 2019.

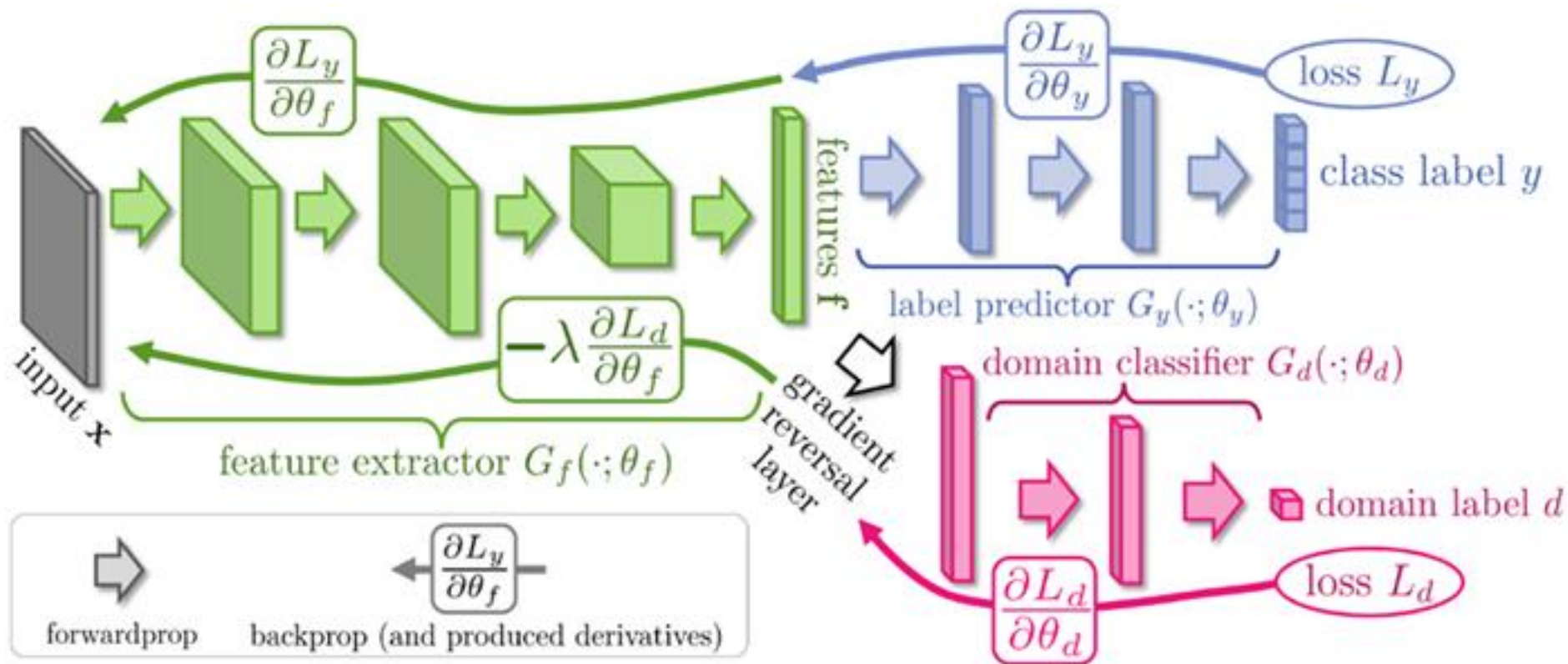
More deep transfer learning can be found at <http://github.com/jindongwang/transferlearning>



### 3. 迁移学习方法

#### □ 基于特征的迁移方法：

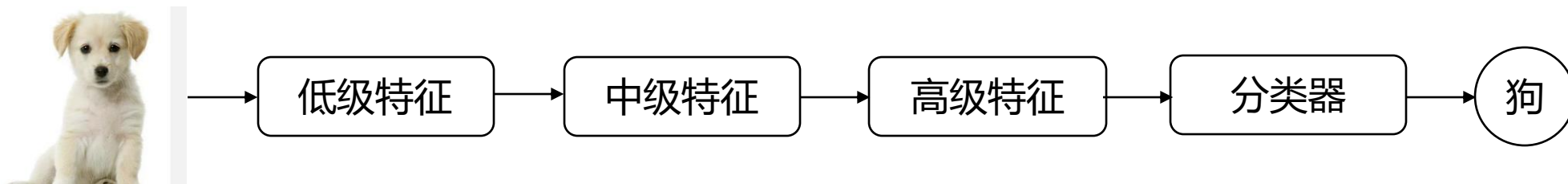
- 隐式度量——例如基于生成对抗网络的迁移学习



### 3. 迁移学习方法



#### □ 基于模型的迁移方法：



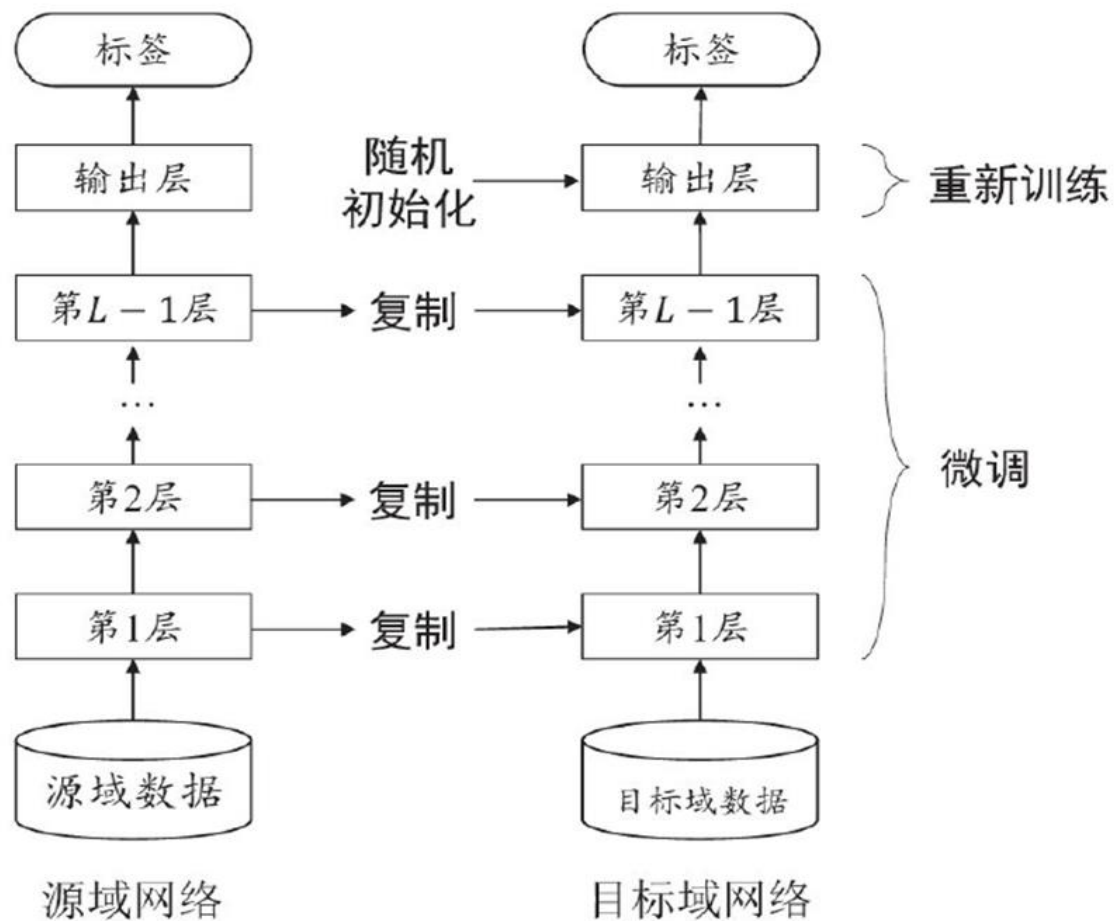
#### ➤ 深度网络的可迁移性

- 网络浅层负责学习通用特征 (General features, 在上图中则为动物边边角角的低级特征)
- 深层则负责学习与任务相关的特殊特征 (Specific features, 在上图中则为腿、脸等中高级特征)
- 随着层次的加深, 网络渐渐从通用特征过渡到特殊特征的学习表征
- 在大数据集上训练得到一个具有强泛化能力的模型 (预训练模型)。对学习通用特征的浅层直接保留并微调, 对学习特殊特征的深层进行改造, 这就是“预训练-微调”

### 3. 迁移学习方法



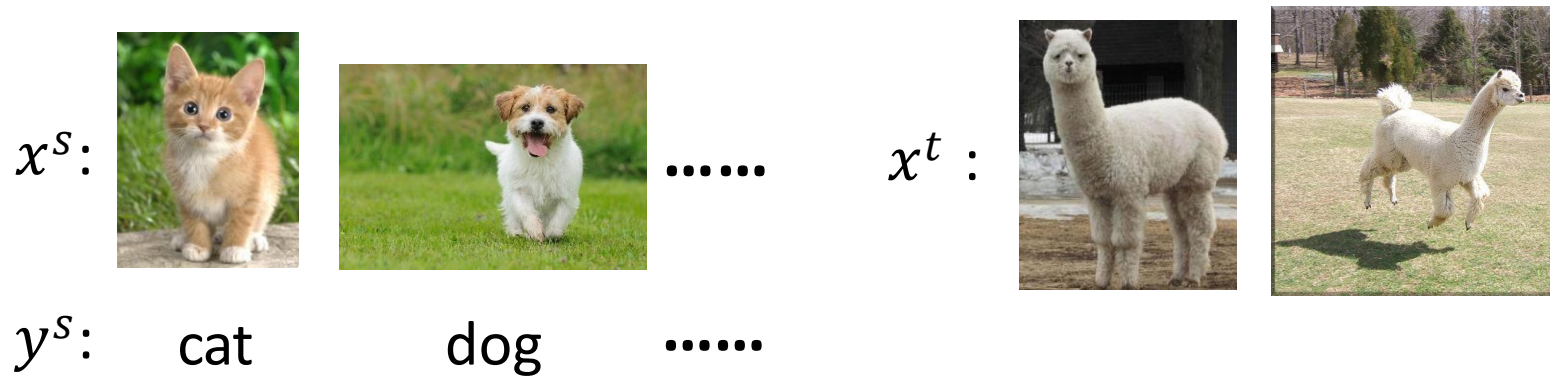
#### □ 基于模型的迁移方法：



### 3. 迁移学习方法

#### ➤ 零样本学习

- Source data:  $(x^s, y^s)$  → Training data
  - Target data:  $(x^t)$  → Testing data
- 不同任务

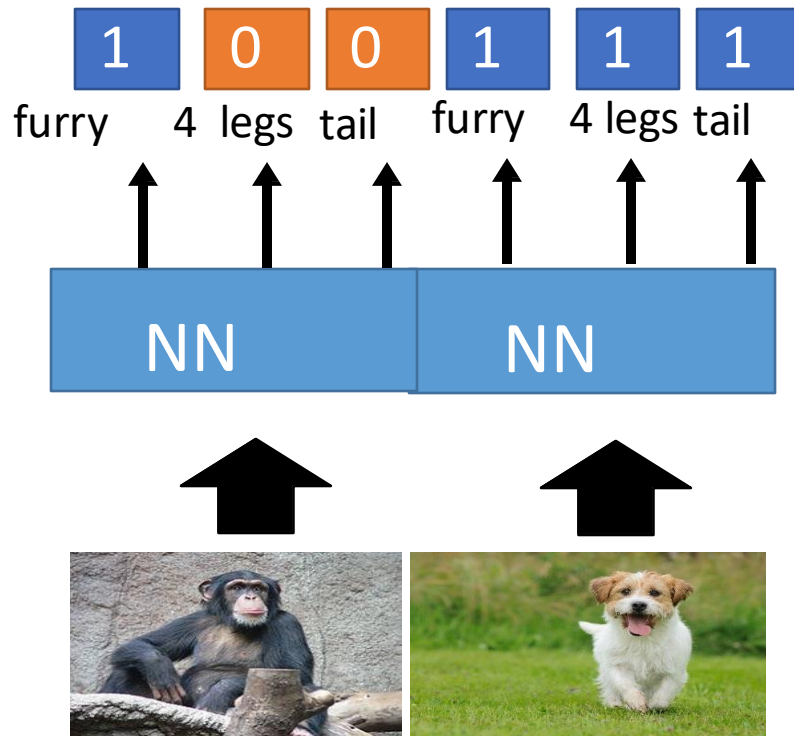


在语音识别中，source(training) data不可能出现所有的词汇。  
我们如何在语音识别中解决这个问题？

### 3. 迁移学习方法

#### ➤ 零样本学习-通过属性来表示每个类

##### Training



##### Database

##### attributes

class

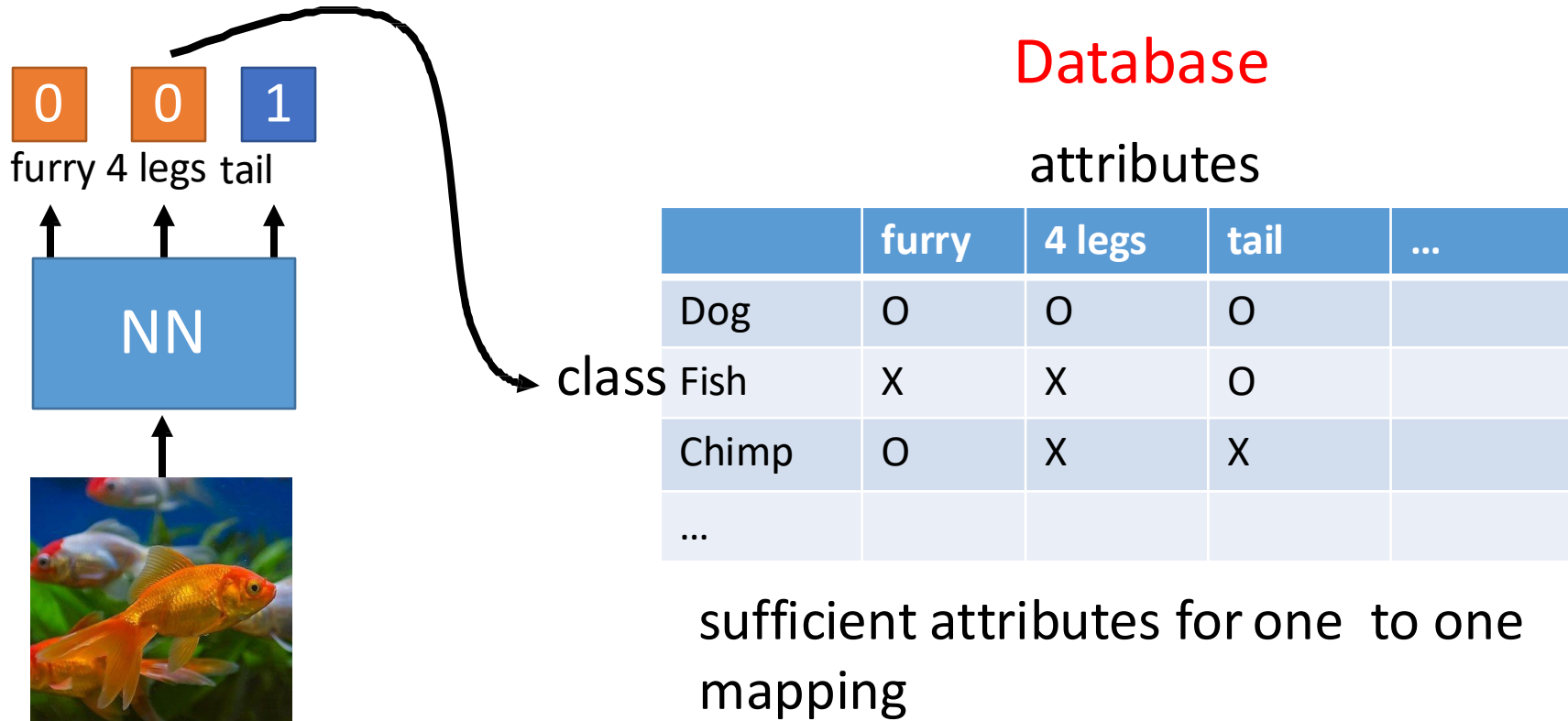
|       | furry | 4 legs | tail | ... |
|-------|-------|--------|------|-----|
| Dog   | O     | O      | O    |     |
| Fish  | X     | X      | O    |     |
| Chimp | O     | X      | X    |     |
| ...   |       |        |      |     |

sufficient attributes for one to one mapping

### 3. 迁移学习方法

#### ➤ 零样本学习-通过属性来表示每个类

##### Testing





### 3. 迁移学习方法

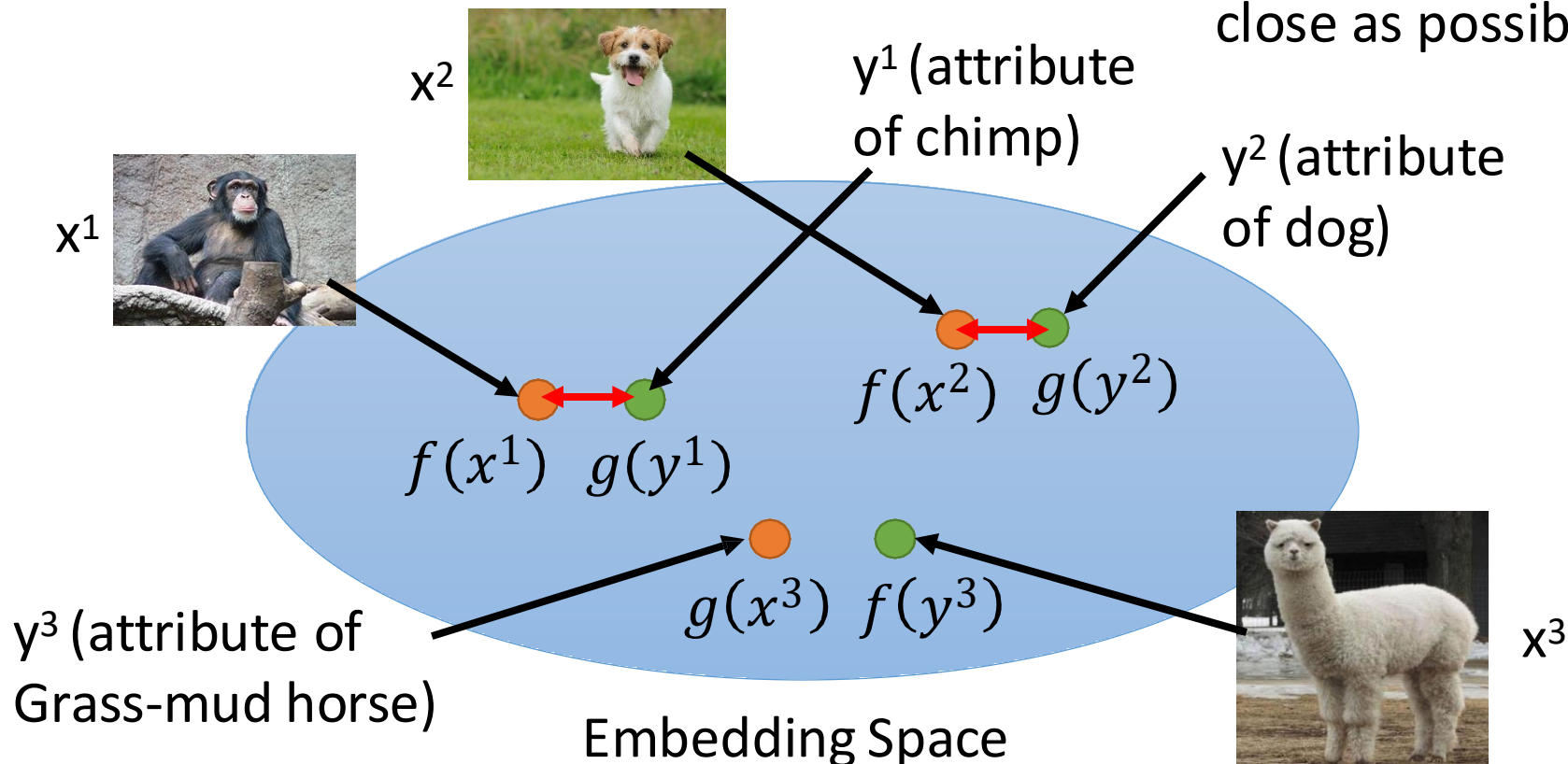
#### ➤ 零样本学习-属性嵌入

$f(*)$  and  $g(*)$  can be NN.



Training target:

$f(x^n)$  and  $g(y^n)$  as close as possible

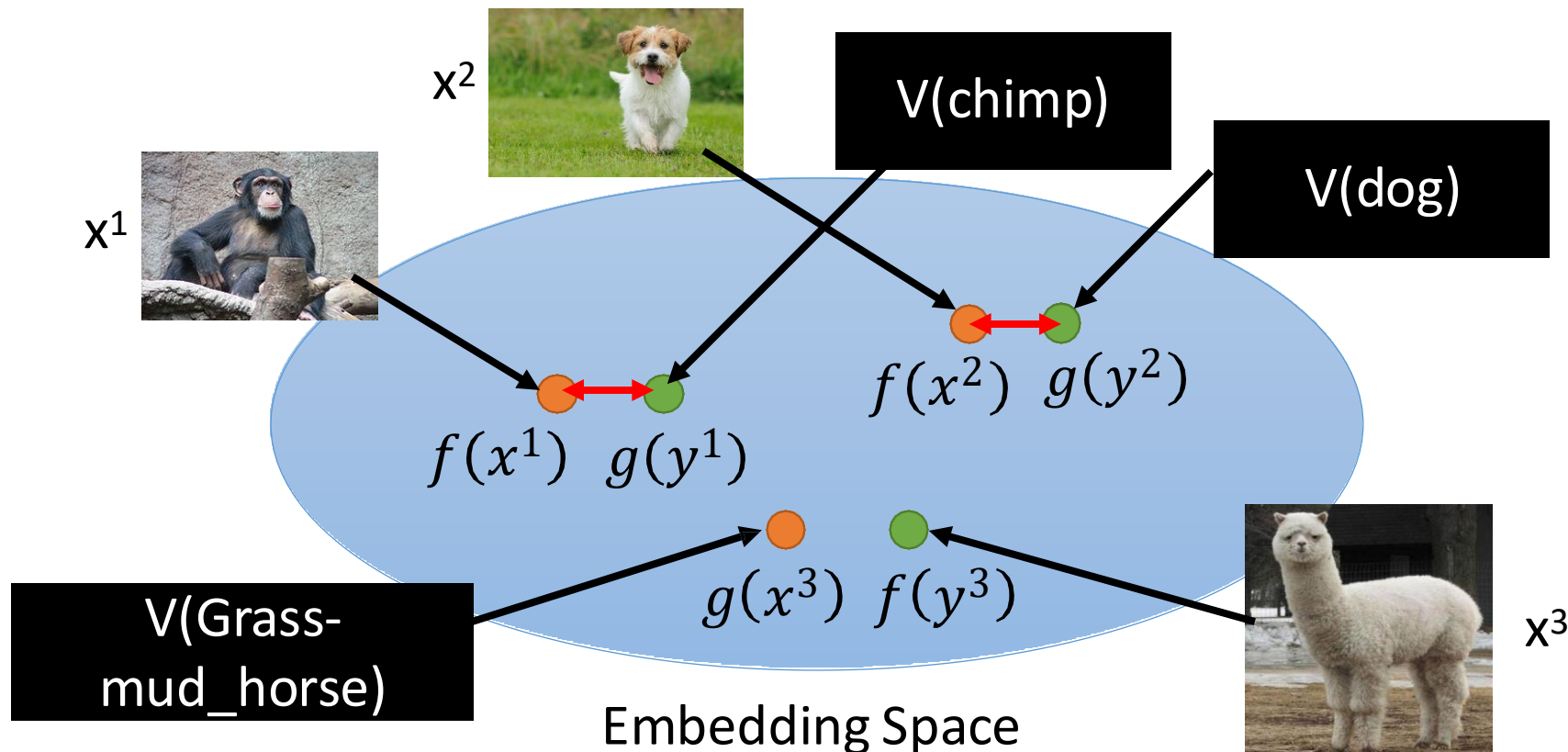




### 3. 迁移学习方法

#### ➤ 零样本学习-属性嵌入+词汇嵌入

如果我们没有数据库  
会怎样



### 3. 迁移学习方法



#### ➤ 零样本学习

$$f^*, g^* = \arg \min_{f, g} \sum_n ||f(x^n) - g(y^n)|| \quad \text{Problem? 因为可能均收敛至0}$$

$$f^*, g^* = \arg \min_{f, g} \sum_n \max(0, k - f(x^n) \cdot g(y^n) + \max_{m \neq n} f(x^n) \cdot g(y^m))$$

Margin you defined      考虑相同样本, 同时考虑不同样本, 以改进其收敛至0的问题

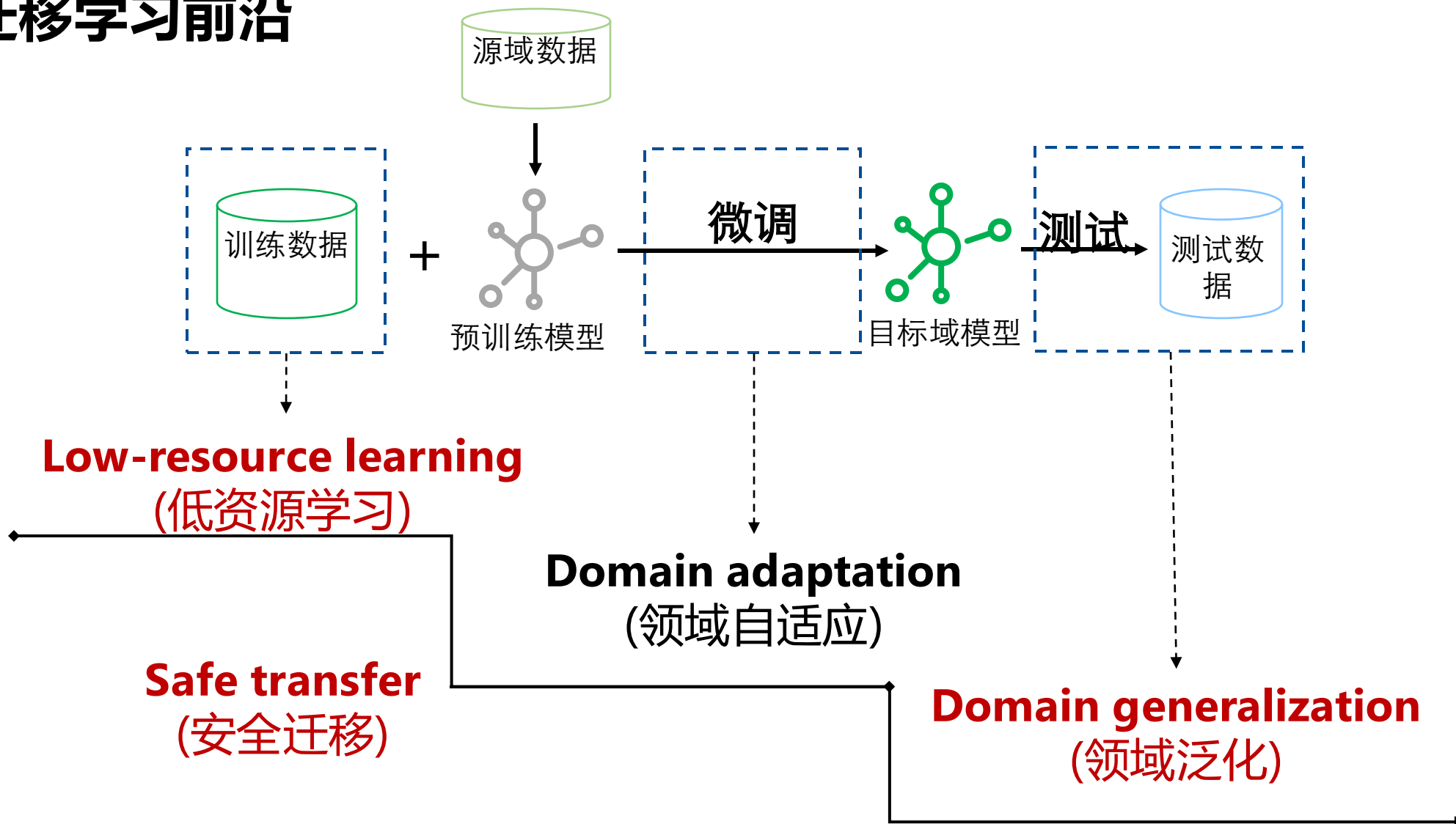
$$\text{Zero loss: } k - f(x^n) \cdot g(y^n) + \max_{m \neq n} f(x^n) \cdot g(y^m) < 0$$

$$\frac{f(x^n) \cdot g(y^n)}{\quad} - \frac{\max_{m \neq n} f(x^n) \cdot g(y^m)}{\quad} > k$$

$f(x^n)$ 和 $g(y^n)$ 越近越好

$f(x^n)$ 和 $g(y^n)$ 要足够远

## 4. 迁移学习前沿



## 4. 迁移学习前沿



### □ 低资源学习:



- Labeled class +1
- Labeled class -1
- Unlabeled samples

- 研究背景
  - 依赖少量有标签样本学习泛化性强的模型
- 研究问题
  - 如何保证网络学习知识从有标签的数据运用于无标签的数据时能保持相同效果?
  - 迁移准则: 固定阈值(Google提出的 FixMatch<sup>[NeurIPS' 20]</sup> )

$$\frac{1}{\mu B} \sum_{b=1}^{\mu B} \mathbb{1}(\max(p_m(y|\omega(u_b))) > \tau) H(\hat{p}_m(y|\omega(u_b)), p_m(y|\Omega(u_b)))$$

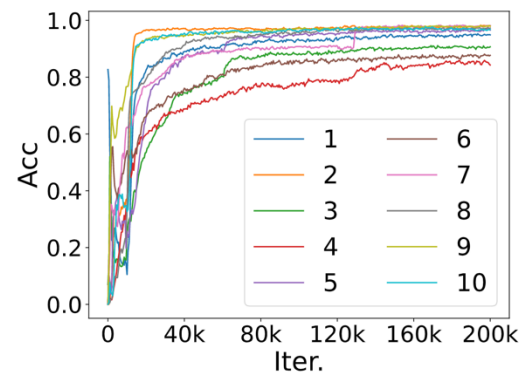
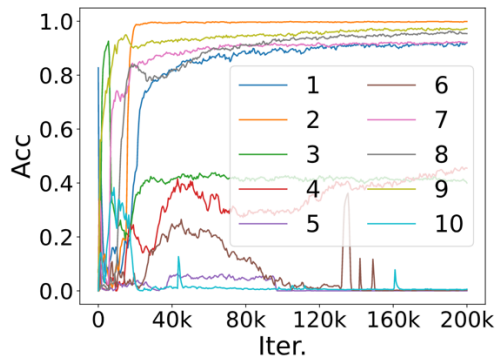
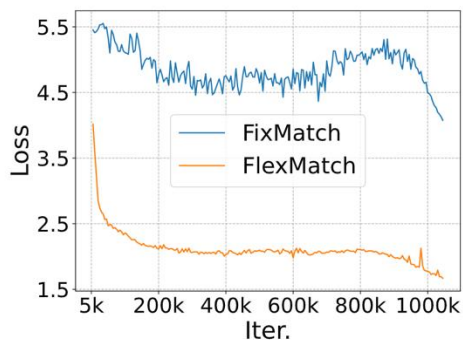
- 研究挑战
  - 预定义阈值能否解决半监督问题?
  - 能否为半监督学习设计更好的阈值计算方式?

## 4.迁移学习前沿



### □低资源学习:

- 固定阈值 vs 可变阈值
  - 结论: 对于不同类别应学习不同阈值



FlexMatch

- 新方法: FlexMatch
  - 对不同类别采用不同阈值→ 类别自适应机制
  - 降低不易学习的类别的阈值→ 学习困难的类别激励机制
  - 如果学习效果好则提高阈值→ 提高要求提升最终准确率表现
  - 在每个时间步骤中为每个类动态调整阈值→ 自动调整机制
- B. Zhang et al. Flexmatch: Boosting semi-supervised learning with curriculum pseudo labeling. NeurIPS 2021. <https://arxiv.org/abs/2110.08263>

## 4. 迁移学习前沿

### □ 领域泛化:

- 研究背景
  - 对多种分布进行训练来学习在未知域上的泛化模型
- 研究问题
  - 解决数据特性随时间变化的问题
  - 解决数据分布是动态变化的问题
- 研究挑战
  - 如何捕捉获取数据分布的动态变化?
  - 如何对时间序列中的数据分布进行量化?
- Wang et al. Generalizing to unseen domains: a survey on domain generalization. IJCAI 2021 survey track.  
<https://arxiv.org/abs/2103.03097>

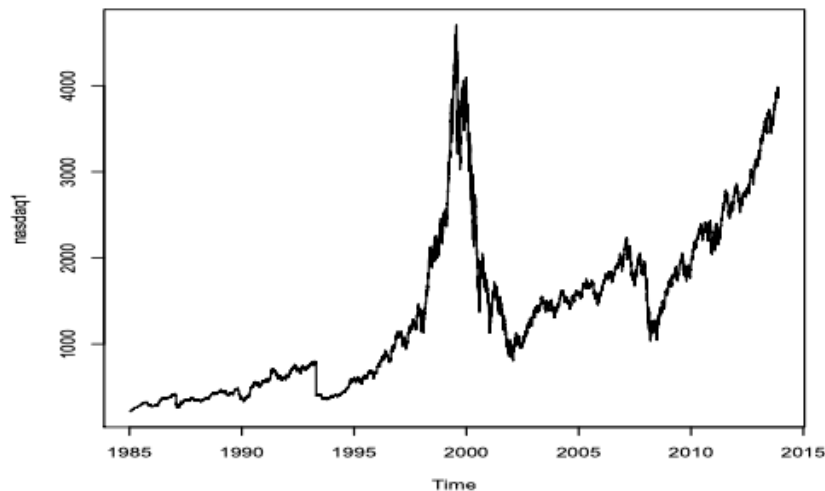
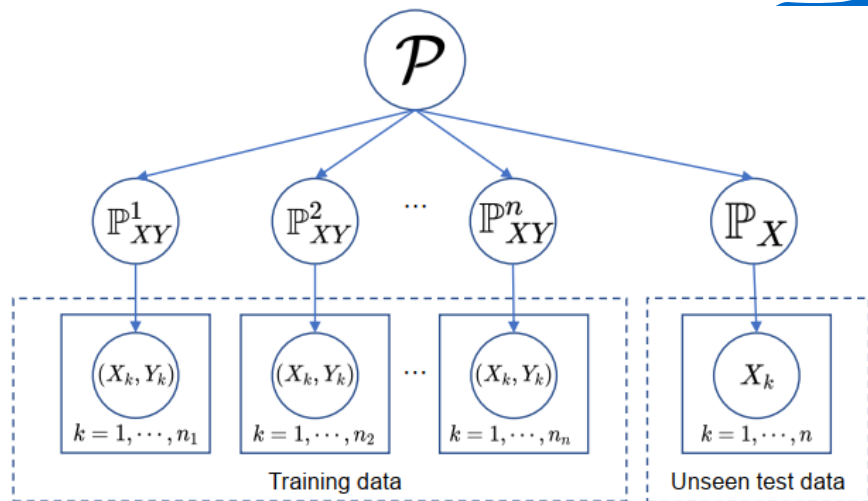


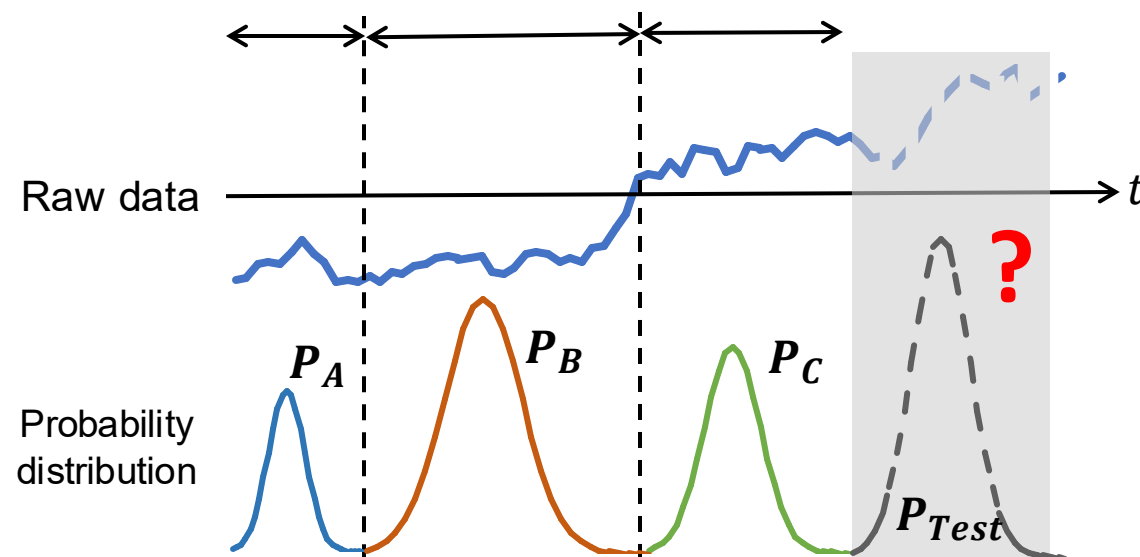
Figure 1.3: Plot of daily closing price of Nasdaq 1985-2014

## 4. 迁移学习前沿



### □ 领域泛化:

- 新方法: AdaRNN (adaptive RNNs)
  - 针对时间协变量漂移问题(TCS)
  - 为解决TCS问题提出了一个可用于连续时序数据的框架



**Temporal Covariate Shift:**  $P_A \neq P_B \neq P_C \neq P_{Test}$

Du Y, Wang J, Feng W, et al. Adarnn: Adaptive learning and forecasting of time series[C]//Proceedings of the 30th ACM International Conference on Information & Knowledge Management. 2021: 402-411. <https://dl.acm.org/doi/abs/10.1145/3459637.3482315>

## 4.迁移学习前沿



### □安全迁移:

#### • 研究动机

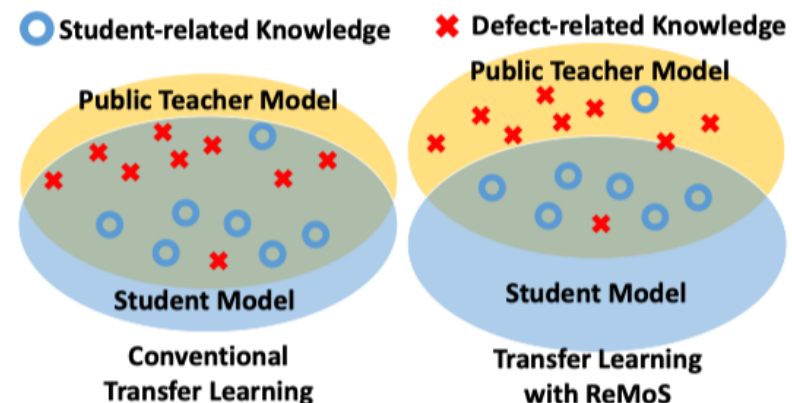
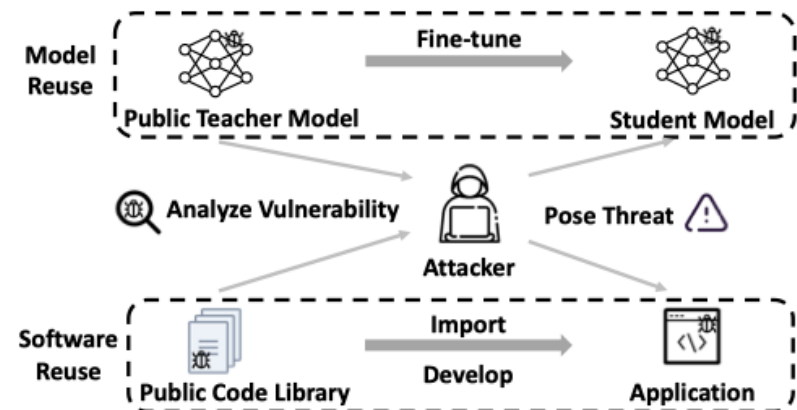
- 软件重复使用在软件工程中非常流行→预训练模型在机器学习中的广泛应用
- 恶意程序/代码会造成损害→微调后的模型可能会预训练模型中继承漏洞
- 缺陷很容易从预训练模型传播给微调后模型，遗传率在52.58% ~ 97.85%之间

#### • 研究问题

- 减少微调后模型从预训练模型继承缺陷
- 同时还要保留它的性能优势

#### • 挑战

- 对模型的攻击方式位置
- 深度神经网络模型多样化且缺乏可解释性





### □安全迁移:

- 新方法: ReMoS— Relevant Model Slicing

- ReMoS给定一个深度神经网络模型 $M$ 和一个目标域数据集 $D$ , ReMoS将计算一个模型权重子集, 该子集与 $D$ 中的样本推断更相关(有一个阈值), 而与 $D$ 之外的样本相关性较小.
- Relevant slicing机制: 从传统的软件工程收到启发
- 相关假设:
  - 预训练模型作为白箱模型, 机理已知 (即架构和权重)
  - 下游任务的目标数据集已知
- 基本方法:

$$\max_{\mathbf{w} \subset \mathbf{w}^T} \sum_{(x,y) \in D^S} \mathbb{I}[f(x; T(\mathbf{w})) = y] + \mathbb{I}[f(\tilde{x}; T(\mathbf{w})) = y]$$

- Zhang et al. ReMoS: reducing defect inheritance in transfer learning by relevant model slicing. ICSE 2022.

```
1 read( a, b )
2 int x=0, y=0
3 x = a + 1
4 y = b + 1
5 int w = 0
6 if x > 3 then
7     if y > -3 then
8         w = w / b
9     endif
10 endif
11 if y > 5 then
12     w = w + 1
13 endif
14 write( w )
```

```
1 read( a, b )
2 int x=0, y=0
3 x = a + 1
4
5 int w = 0
6 if x > 3 then
7
8
9
10 endif
11 if y > 5 then
12     w = w + 1
13 endif
14 write( w )
```

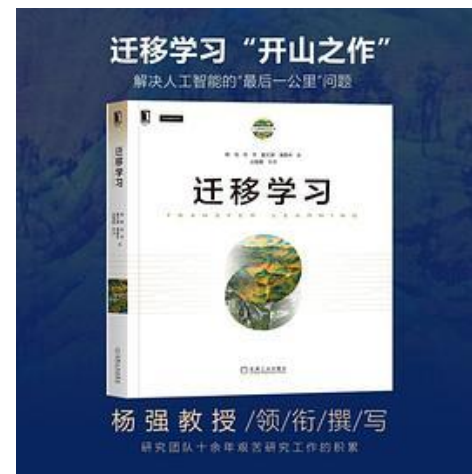
## 5.相关读物推荐



### □迁移学习经典书籍：



书名：迁移学习导论（第二版）  
作者：王晋东，陈益强 著  
出版社：电子工业出版社



书名：迁移学习  
作者：杨强，张宇等 著  
出版社：机械工业出版社